

09- Performance Parallel testing

1. What is performance testing?



Performance

- How **fast** are operations performed by the system under certain **load**?
- Examples for Operations:
 - **Submitting data** after clicking a button on an HTML page
 - **REST API call**
 - **Disk read / write** operation
- Different tasks imply different performance expectations

Performance Testing Metrics: Parameters Monitored - 1

- **Processor Usage** – an amount of time processor spends executing non-idle threads.
- **Memory use** – amount of **physical** memory available to processes on a computer.
- **Disk time** – amount of time disk is busy executing a read or write request.
- **Bandwidth** – shows the bits per second used by a network interface.
- **Private bytes** – number of bytes a process has allocated that **can't** be shared amongst other processes. These are used to measure memory leaks and usage.
- **Committed memory** – amount of **virtual** memory used.
- **Network bytes total per second** – rate which bytes are sent and received on the interface including framing characters.
- **Response time** – time from when a user enters a request until the first character of the response is received.

<https://www.guru99.com/performance-testing.html>

Performance Testing Metrics: Parameters Monitored - 2

- **Memory pages/second** – number of pages written to or read from the disk in order to resolve hard page faults. Hard page faults are when code not from the current working set is called up from elsewhere and retrieved from a disk.
- **Page faults/second** – the overall rate in which fault pages are processed by the processor. This again occurs when a process requires code from outside its working set.
- **CPU interrupts per second** – is the avg. number of hardware interrupts a processor is receiving and processing each second.
- **Disk queue length** – is the avg. no. of read and write requests queued for the selected disk during a sample interval.
- **Network output queue length** – length of the output packet queue in packets. Anything more than two means a delay and bottlenecking needs to be stopped.

<https://www.guru99.com/performance-testing.html>

Performance Testing Metrics: Parameters Monitored - 3

- **Throughput** – rate a computer or network receives requests per second.
- **Amount of connection pooling** – the number of user requests that are met by **pooled** connections قنوات سابقة التحضير. The more requests met by connections in the pool, the better the performance will be.
- **Maximum active sessions** – the maximum number of sessions that can be active at once.
- **Hit ratios** – This has to do with the number of SQL statements that are handled by **cached** data instead of expensive I/O operations. This is a good place to start for solving bottlenecking issues.
- **Hits per second** – the no. of hits on a web server during each second of a load test.

Performance Testing Metrics: Parameters Monitored - 4

- **Rollback segment** – the amount of data that can **rollback** يتم التراجع عنها at any point in time.
- **Database locks** – **locking** of tables and databases needs to be monitored and carefully tuned. عدد مرات منع الدخول لأكثر من مستخدم على نفس العنصر
- **Thread counts** – An applications health can be measured by the no. of threads that are running and currently active.
- **Garbage collection** – It has to do with returning unused memory back to the system. Garbage collection needs to be monitored for efficiency.

Example Performance Test Cases

- **Test Case 01:** Verify **response time is not more than 4 secs** when 1000 users access the website simultaneously.
- **Test Case 02:** Verify response time of the Application Under Load is within an acceptable range when the network connectivity is slow
- **Test Case 03:** Check the **maximum number of users** that the application can **handle before it crashes**.
- **Test Case 04:** Check database execution time when 500 records are read/written simultaneously.
- **Test Case 05:** Check **CPU and memory usage** of the application and the database server under peak load conditions
- **Test Case 06:** Verify the **response time** of the application under **low, normal, moderate**, and heavy load conditions.

Key Performance Metrics

- Response time
- Latency
- Throughput
- IOPS
- Utilization
- Saturation

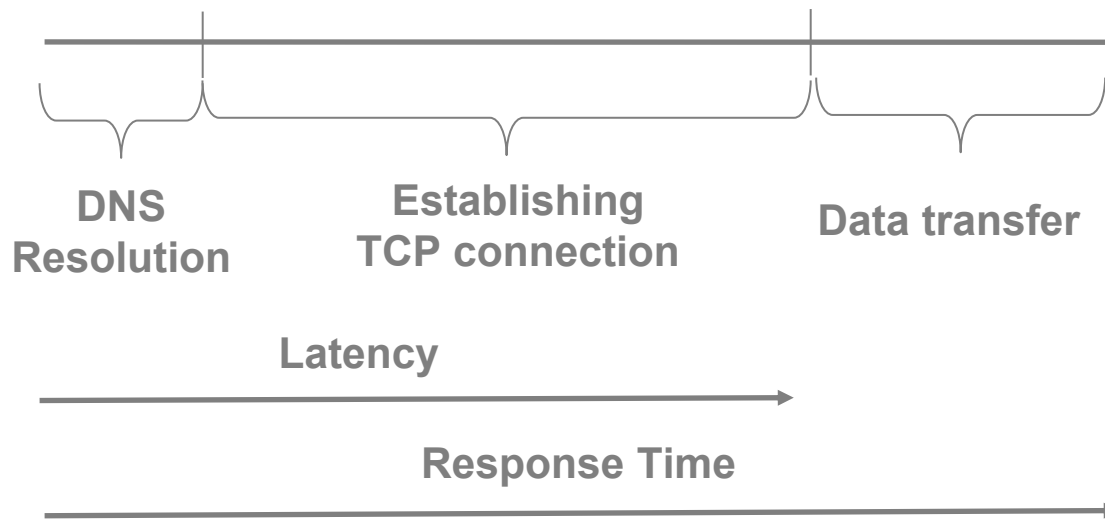
Response Time and Latency

- **Response time** = time for an operation **to complete**, **including** the time spent **waiting** and being **served**
- **Latency** = time an operation spent **waiting** in a **queue**
- **Service time** = **Actual time** to perform job continuously.
- **Response time = Latency + Service time**
- Easily quantifying degradation and improvement
- Interpretation depends on the purpose of the operation

Latency - HTTP GET Request

- Assume that a GET request consists of:
 - DNS resolution
 - Establishing TCP connection
 - Data transfer from the server to the client
- What do we mean by latency and response time?

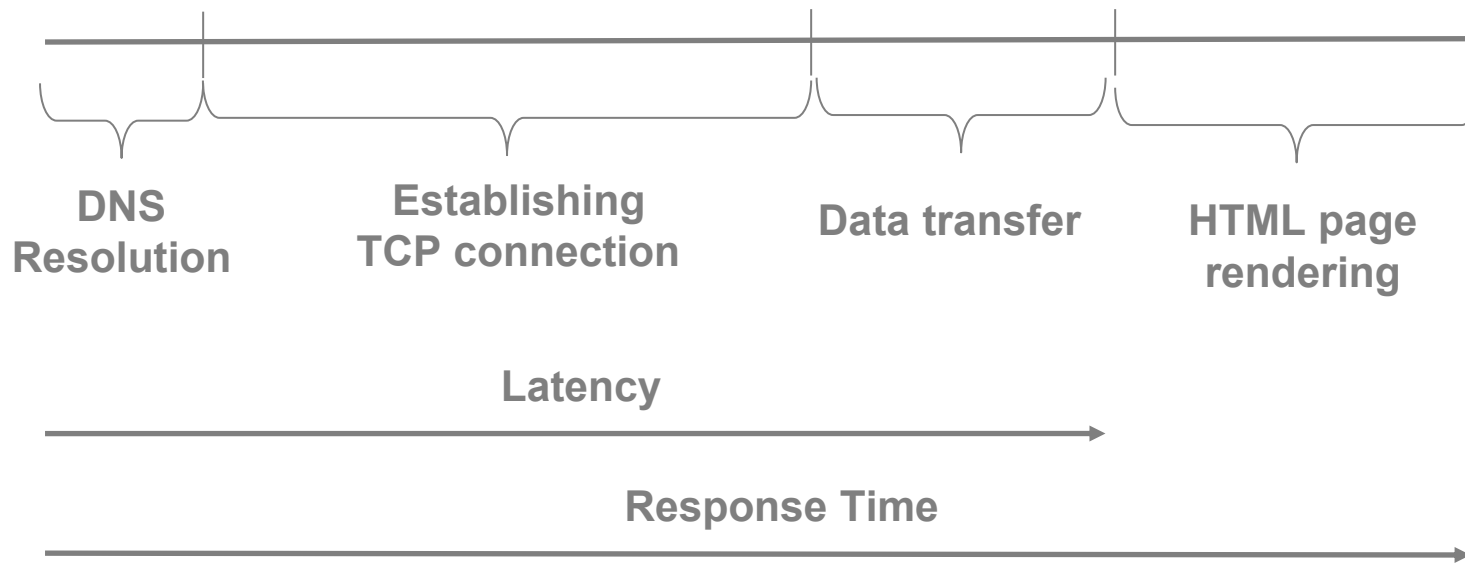
Latency – HTTP GET Request (2)



Latency – HTML Rendering

- User submits the data entered into a form on HTML page and waits until the page is rendered with the server response.
- What do we mean by latency and response time?

Latency – HTML Rendering (2)



Throughput

- Throughput is a measure of how many **units of information** a system can process in a given **amount of time (Second)**.
- The rate at which work is completed
- The meaning depends on the target evaluated
- Example measures:
 - Bytes / bits per second
 - SQL queries per second
 - REST calls per second
 - Transactions per second

IOPS

- Input / output operations completed per second
- Throughput-oriented metric
- The meaning depends on the target evaluated
- Examples:
 - Network devices (TCP) – packets received / sent per second
 - Block devices – reads / writes per second

Utilization

There are 2 definitions:

- 1. Time-based definition:** How much **time** a resource is busy during a period of time.
 - CPU Utilization
 - Disk Utilization
 - 100% Utilization doesn't have to lead to the saturation of the resource. We need to examine other metrics of the resource that indicates its saturation
- 2. Capacity-based definition:** The extent to which the capacity مقدار استغلال القدرات المتاحة of a resource was used during a time period.

Note about capacity utilization

- it assumes that every resource (component) has some maximum capacity it can deliver.
- However, it's very **difficult to determine** the maximum capacity for some resources,
- for instance hard disks. To determine the maximum capacity of hard disks, the other parts of computer systems that disks use must be evaluated.

Saturation

- The extent to which a resource has **queued** work because it **cannot accept** more work **التعطل بسبب الوصول للذروة**
- Capacity-based Utilization $\geq 100\%$
- Increases latency and response time
- **Bottleneck** = the **resource** that **limits** performance

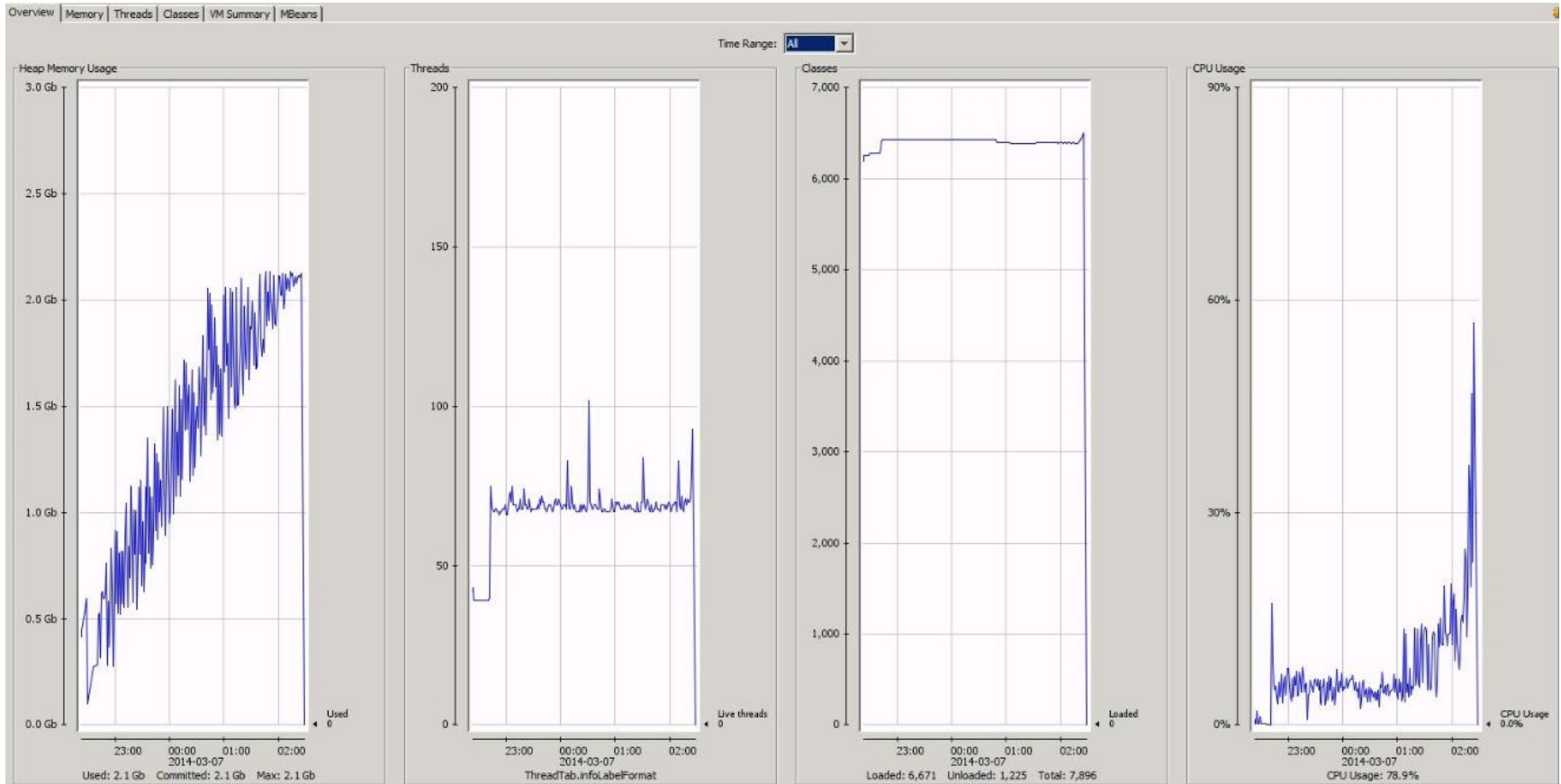
Scalability

- How does the performance of the product **change** under **increasing load**?
- **Load** meanings examples:
 - Hourly number of users accessing the product
 - API calls per second
 - Amount of data uploaded to the product per second
- System may perform in normal load but may not be scalable

Stability

- Does the system perform over **long time**?
- Does performance gets back to normal after an exceptional situation?
- **Memory leaks** (objects created and no more required, but still in memory)
- Excessive logging on disk devices

Memory Leak



More Types of Performance Testing

- **Spike testing** – tests the software's reaction to **sudden large spikes** ارتفاع مفاجئ في الأحمال in the load generated by users.
- **Volume testing** – Under Volume Testing **large number of Data** is populated in a database, and the overall software system's behavior is monitored. The objective is to check software application's performance under varying database volumes.

<https://www.guru99.com/performance-testing.html>

Part 2- Performance Testing with JMeter

Based on internet slides prepared By Mohd Nazim



Jmeter Overview

- JMeter is a software that can perform load test, performance test, business (functional) test, regression test, etc., on different protocols or technologies
- JMeter is a Java desktop application with a graphical interface . It can therefore run on any environment / workstation that accepts a Java virtual machine, for example: Windows, Linux, Mac, etc.
- Protocols supported by JMeter like:
 1. Web: HTTP, HTTPS ...
 2. Web Services: SOAP / XML-RPC
 3. Service: POP3, IMAP, SMTP
 4. FTP Service

-

Load & Stress Testing

- JMeter Performance Testing includes:
 1. **Load** Testing: Modeling the **expected usage** by simulating multiple user access the Web services concurrently.
 2. **Stress** Testing: Every web server has a maximum load capacity. When the load **goes beyond** تجاوز **الحد الأقصى** the limit, the web server starts responding slowly and produce errors. The purpose of the Stress Testing is to **find the maximum load** the web server can handle.

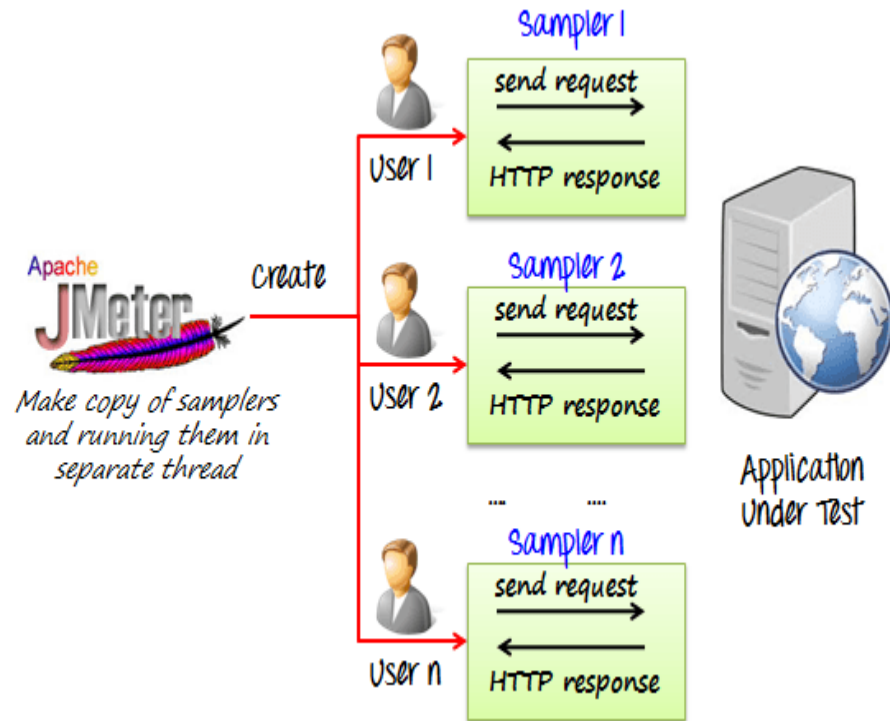
JMeter Features

- **Open source license:** JMeter is totally free, allows developer use the source code for the development
- **Friendly GUI:** JMeter is extremely easy to use and doesn't take time to get familiar with it
- **Platform independent:** JMeter is 100% pure Java desktop application. So it can run on multiple platforms
- **Full multithreading framework.** JMeter allows concurrent and simultaneous sampling of different functions by a separate thread group
- **Visualize Test Result:** Test result can be displayed in a different format such as chart, table, tree and log file
- **Easy installation:** You just copy and run the *.bat file to run JMeter. No installation needed.
- **Highly Extensible:** You can write your own tests. JMeter also supports visualization plugins allow you to extend your testing



JMeter Features (2)

- **Multiple testing strategy:** JMeter supports many testing strategies such as Load Testing, Distributed Testing, and Functional Testing.
- **Simulation:** JMeter can simulate multiple users with concurrent threads, create a heavy load against web application under test
- **Support multi-protocol:** JMeter does not only support web application testing but also evaluate database server performance. All basic protocols such as HTTP, JDBC, LDAP, SOAP, JMS, and FTP are supported by JMeter
- **Record & Playback - Record** the user activity on the browser and simulate them in a web application using JMeter
- **Script Test:** Jmeter can be integrated with other tools like Selenium for automated testing.

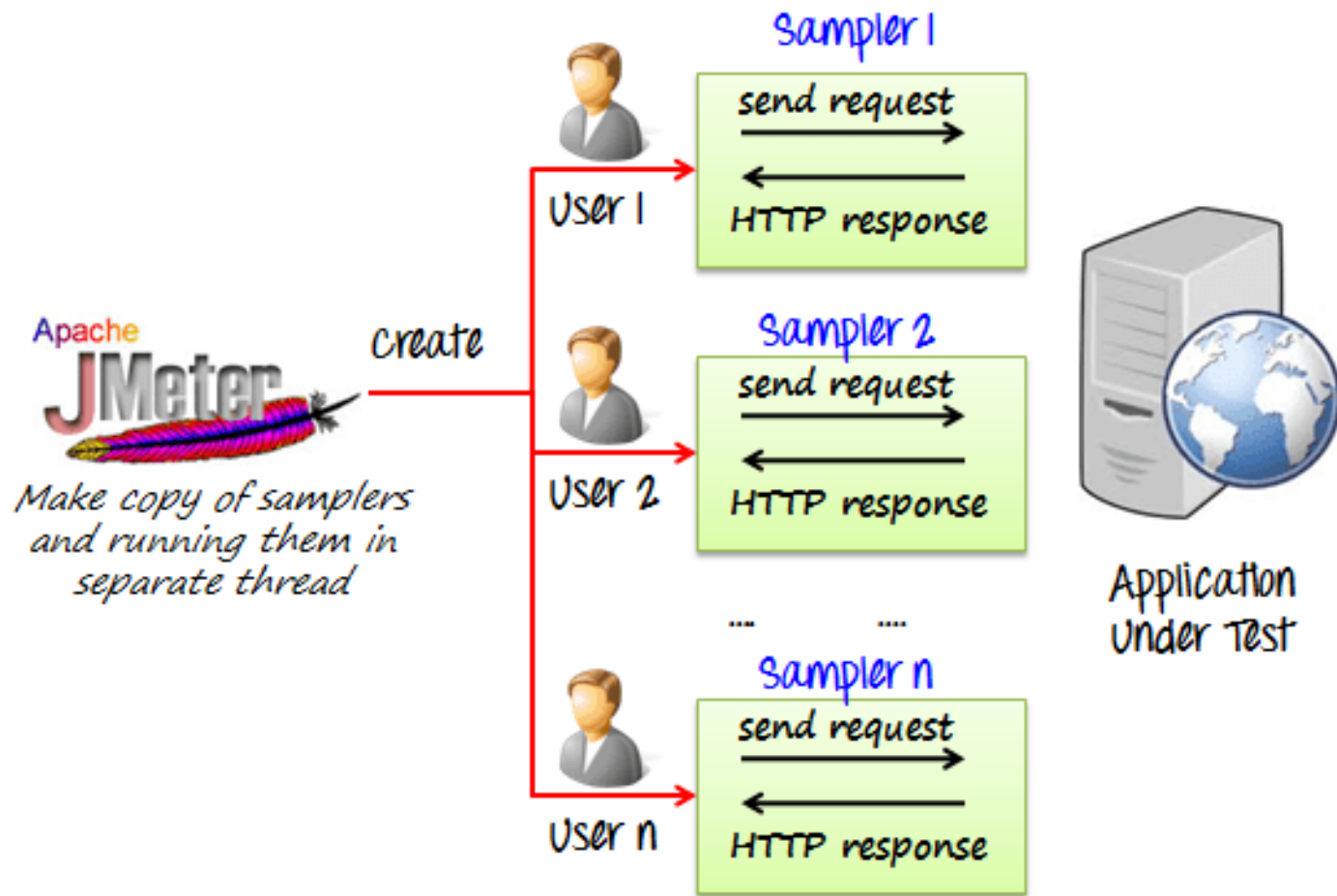


Jmeter Vs Load Runner



Jmeter	HP Load Runner
Open source and free to use	It is a licensed tool and costly
Jmeter is cross platform	It works with Window OS and Linux
It has unlimited load generation capacity	It has limited load generation capacity
It is standalone application	It has 3 separate application
Scripting isn't essential in Jmeter	It requires scripting knowledge
Elements are easier to define in Jmeter	Configuring each element is more complex. They all require complex scripting in C

Jmeter Simulates multiple Users (Threads)

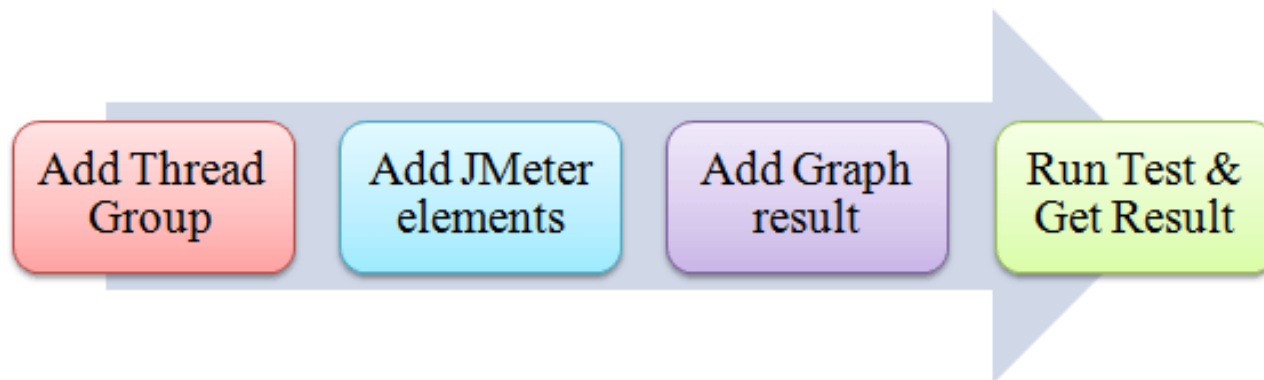


Traditional scenario for using JMeter

Before testing the performance of target web application, we should determine-

- **Normal Load:** Average number of users visit your website
- **Heavy Load:** The maximum number of users visit your website
- What is your **target** in this test?

Here is the **roadmap** of traditional scenario for using jmeter



<https://www.guru99.com/jmeter-performance-testing.html>

JMeter Environment Setup

- JMeter is a framework for Java, so the very first requirement is to have JDK 8+ installed in your machine.
- Good tutorials:
 - <https://howtodoinjava.com/library/jmeter-beginners-tutorial/>
 - <https://www.guru99.com/introduction-to-jmeter.html>
 - <https://www.udemy.com/course/jmeter-step-by-step-for-beginners>
 - <https://www.digitalocean.com/community/tutorials/how-to-use-apache-jmeter-to-perform-load-testing-on-a-web-server>

Steps to create a Script

- **1. Download and start JMeter**
- **2. Create test plan**
- **3. Perform load testing**
- Note: the next tutorial steps are based on <https://howtodoinjava.com/library/jmeter-beginners-tutorial/>

1.Download and start JMeter

- **1.1. Download JMeter**
- Go to Apache [jmeter download page](#) and download the distribution based on your machine.
- Extract the downloaded package to a desired location.

Apache JMeter 5.0 (Requires Java 8 or 9.)

Binaries

[apache-jmeter-5.0.tgz](#) [sha512](#) [pgp](#)

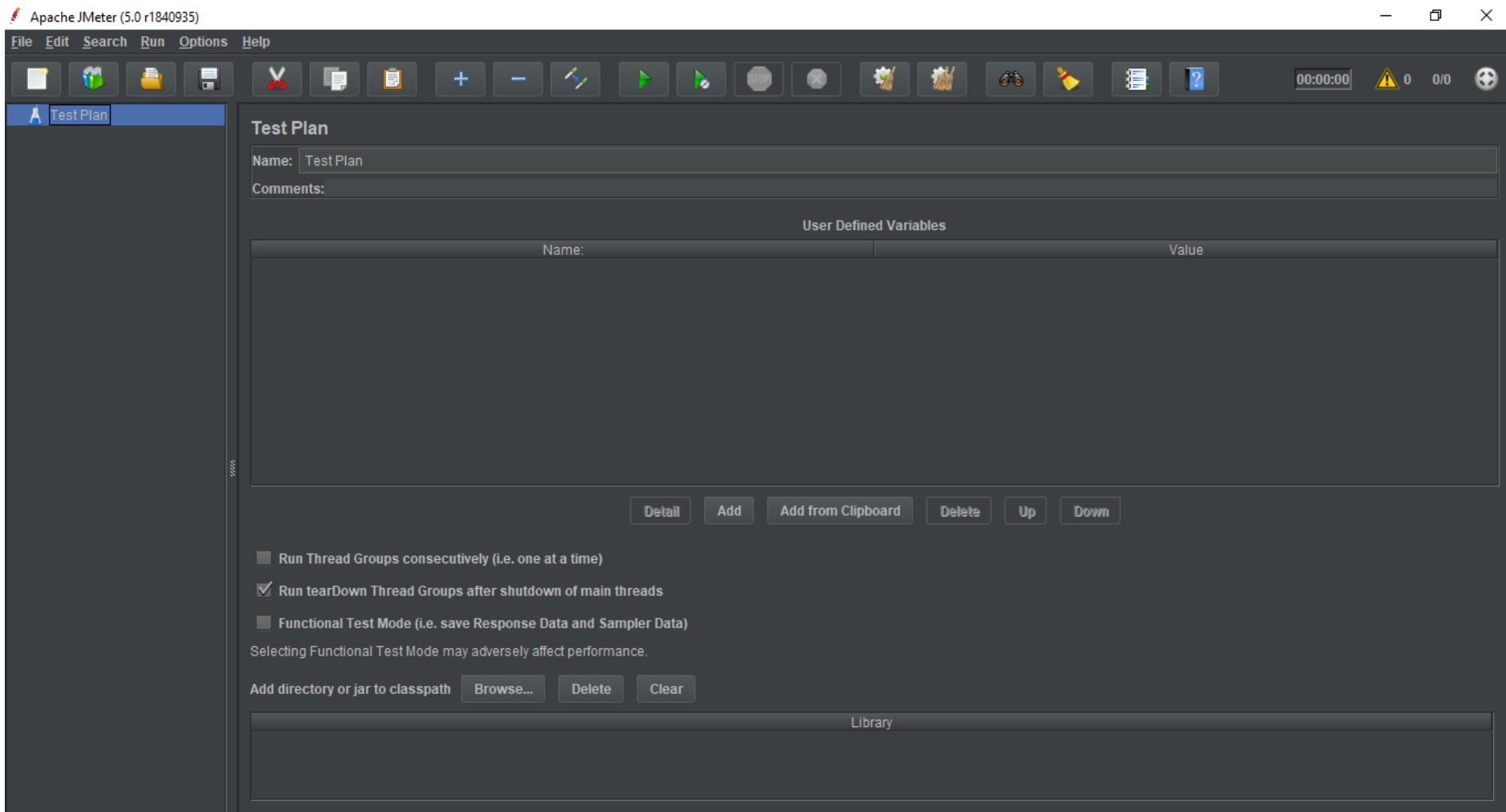
[apache-jmeter-5.0.zip](#) [sha512](#) [pgp](#)

1.2 Start JMeter

- To run the jmeter, go to the folder the /jmeter/bin and run **jmeter.bat** file.

Name	Date modified	Type	Size
examples	11/20/2018 12:13 ...	File folder	
report-template	11/20/2018 12:12 ...	File folder	
templates	11/20/2018 12:13 ...	File folder	
ApacheJMeter	11/20/2018 12:12 ...	Executable Jar File	13 KB
BeanShellAssertion.bshrc	11/20/2018 12:12 ...	BSHRC File	2 KB
BeanShellFunction.bshrc	11/20/2018 12:12 ...	BSHRC File	3 KB
BeanShellListeners.bshrc	11/20/2018 12:12 ...	BSHRC File	2 KB
BeanShellSampler.bshrc	11/20/2018 12:12 ...	BSHRC File	3 KB
create-rmi-keystore	11/20/2018 12:12 ...	Windows Batch File	2 KB
create-rmi-keystore	11/20/2018 12:12 ...	SH Source File	2 KB
hc.parameters	11/20/2018 12:12 ...	PARAMETERS File	2 KB
heapdump	11/20/2018 12:12 ...	Windows Comma...	2 KB
heapdump	11/20/2018 12:12 ...	SH Source File	2 KB
jaas.conf	11/20/2018 12:12 ...	CONF File	2 KB
jmeter	11/20/2018 12:12 ...	File	8 KB
jmeter	11/20/2018 12:12 ...	Windows Batch File	9 KB
jmeter	11/20/2018 1:13 PM	Text Document	30 KB
jmeter	11/20/2018 12:12 ...	Properties Source ...	54 KB
jmeter	11/20/2018 12:12 ...	SH Source File	4 KB

- It will start the JMeter UI with no test plan initially. Now, it's time to create a test plan for our web application load testing.



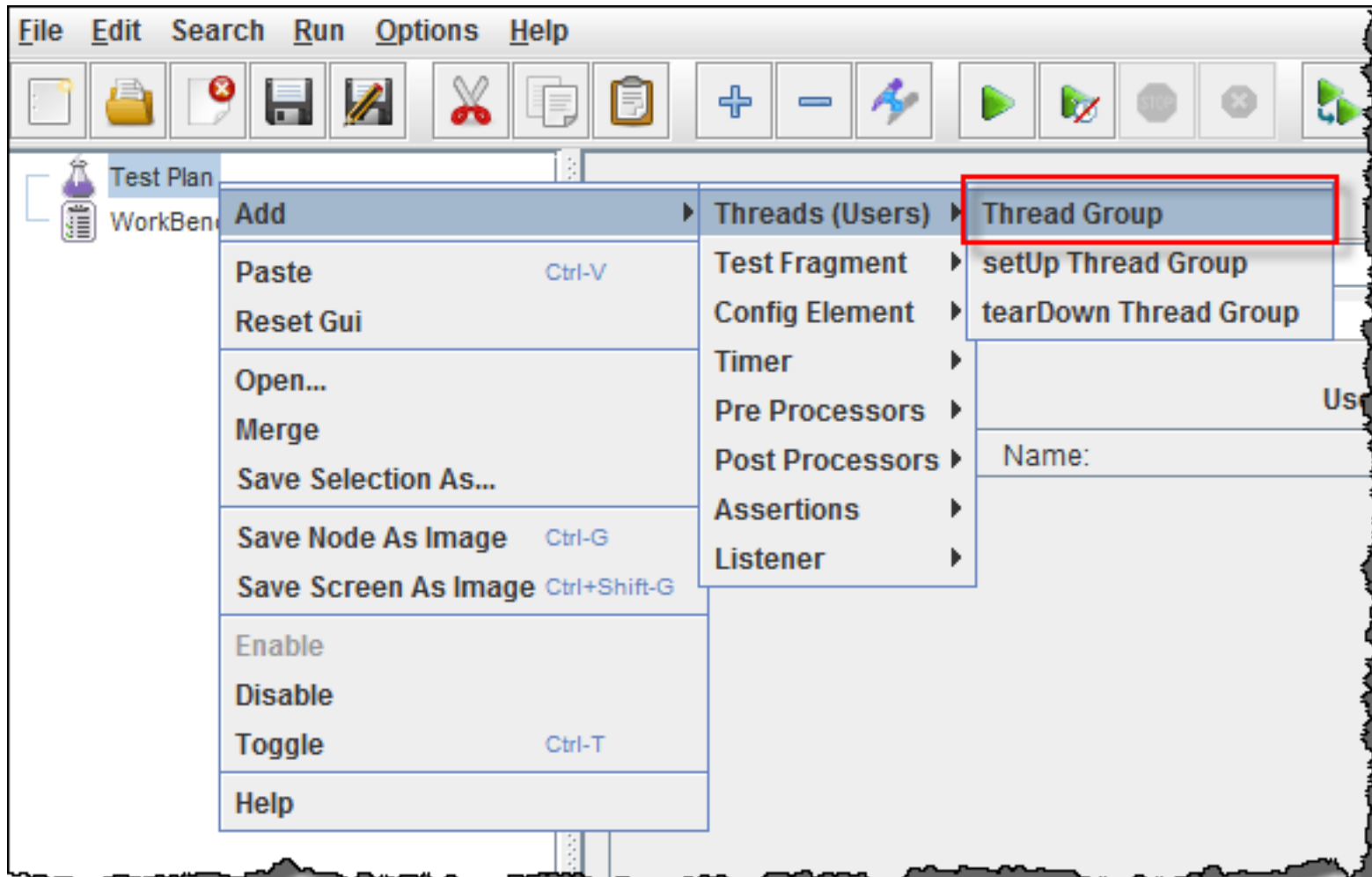
2. Create test plan

A useful test plan is created with minimum 3 components –

- **Thread Group** –
 - contains the simulation of multiple concurrent users.
 - **A single thread represent a single user.**
 - We can create any number of threads to put the desired load on the application.
 - It also help us in scheduling the delay between two threads, and any repetition of request batches.
- **HTTP Request** –
 - consist the HTTP request configuration which thread group will be invoking.
 - It is **the application URL** which you want to load test.
- **Listener** –
 - helps in viewing the **result** of the **whole** testing process.
 - There are multiple listener available in jmeter to verify the testing results.

2.1. Create Thread Group

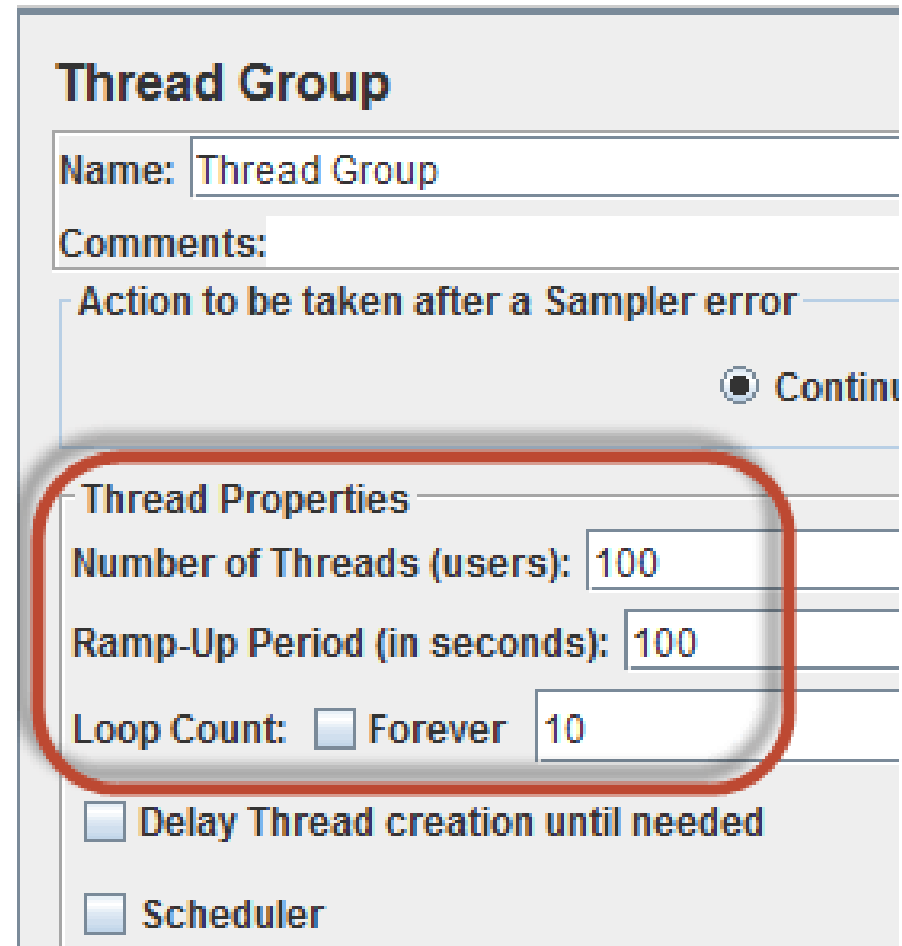
- To create a thread group, navigate to *'Right click Test Plan -> Add -> Threads -> Thread Group'*.



- Fill in the values as per your requirements (or based on your assumptions, we can change them anytime in future).
- Name the thread group and save it to any location in your workstation.

In this Example:

- **Number of Threads:** 100 (Number of users connects to the target website: 100)
- **Loop Count:** 10 (Number of time to execute testing)
- **Ramp-Up Period:** 100



Thread Group

Name: Thread Group

Comments:

Action to be taken after a Sampler error

☒ Continue

Thread Properties

Number of Threads (users): 100

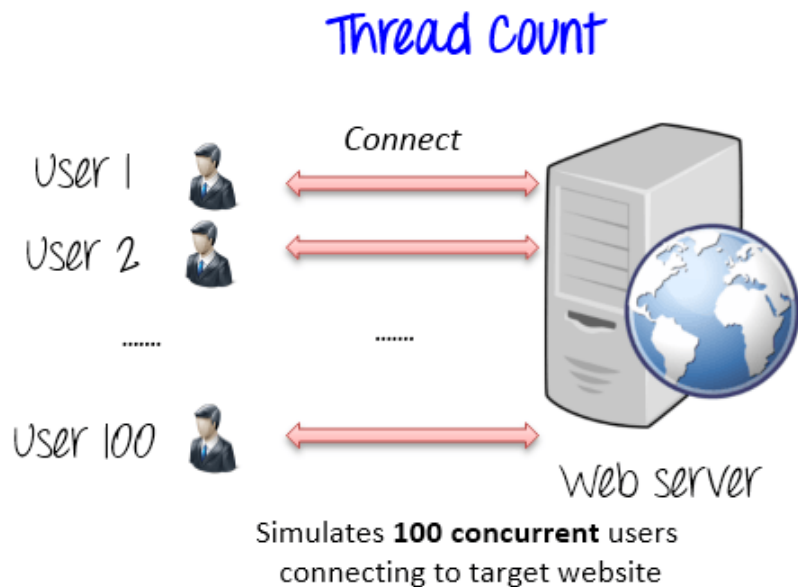
Ramp-Up Period (in seconds): 100

Loop Count: ☐ Forever 10

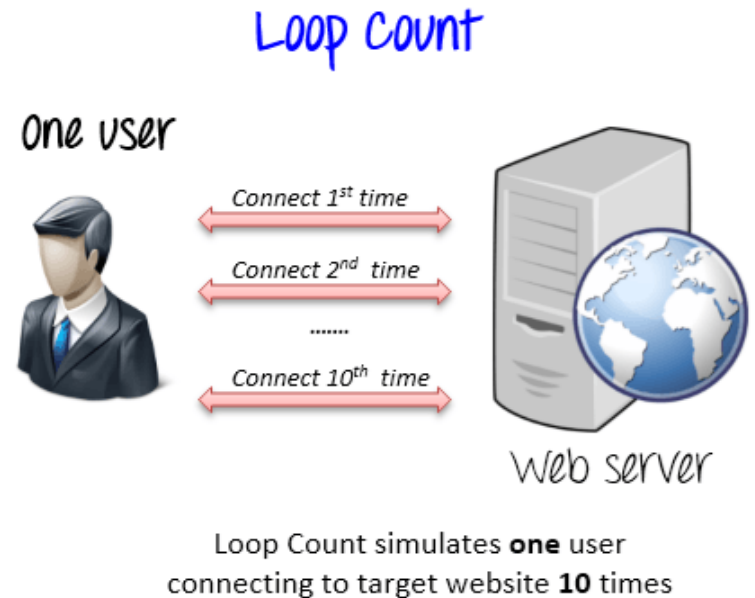
☐ Delay Thread creation until needed

☐ Scheduler

The Thread Count and The Loop Counts are **different**

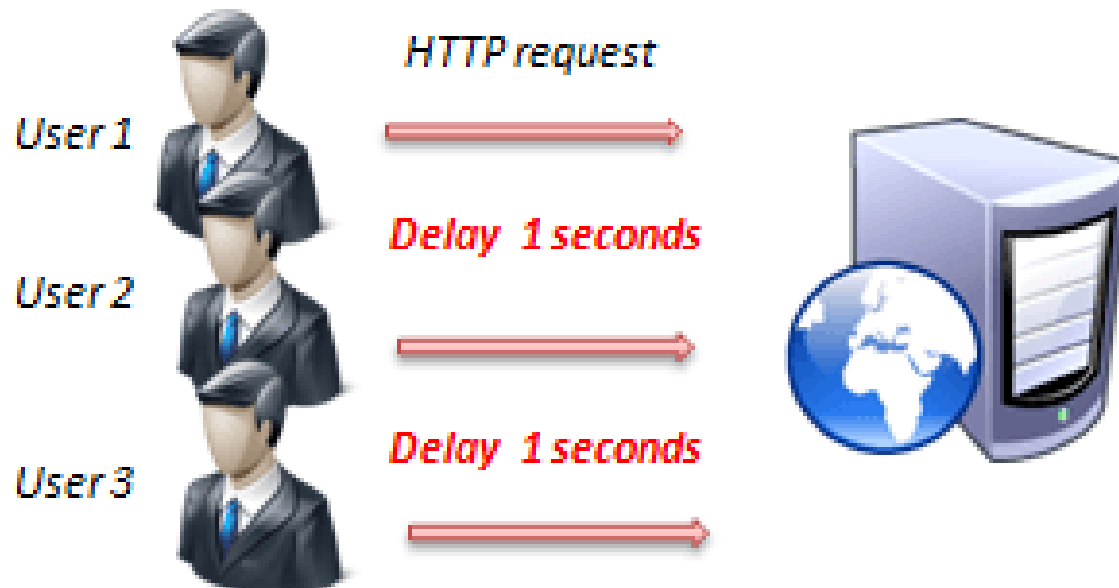


VS



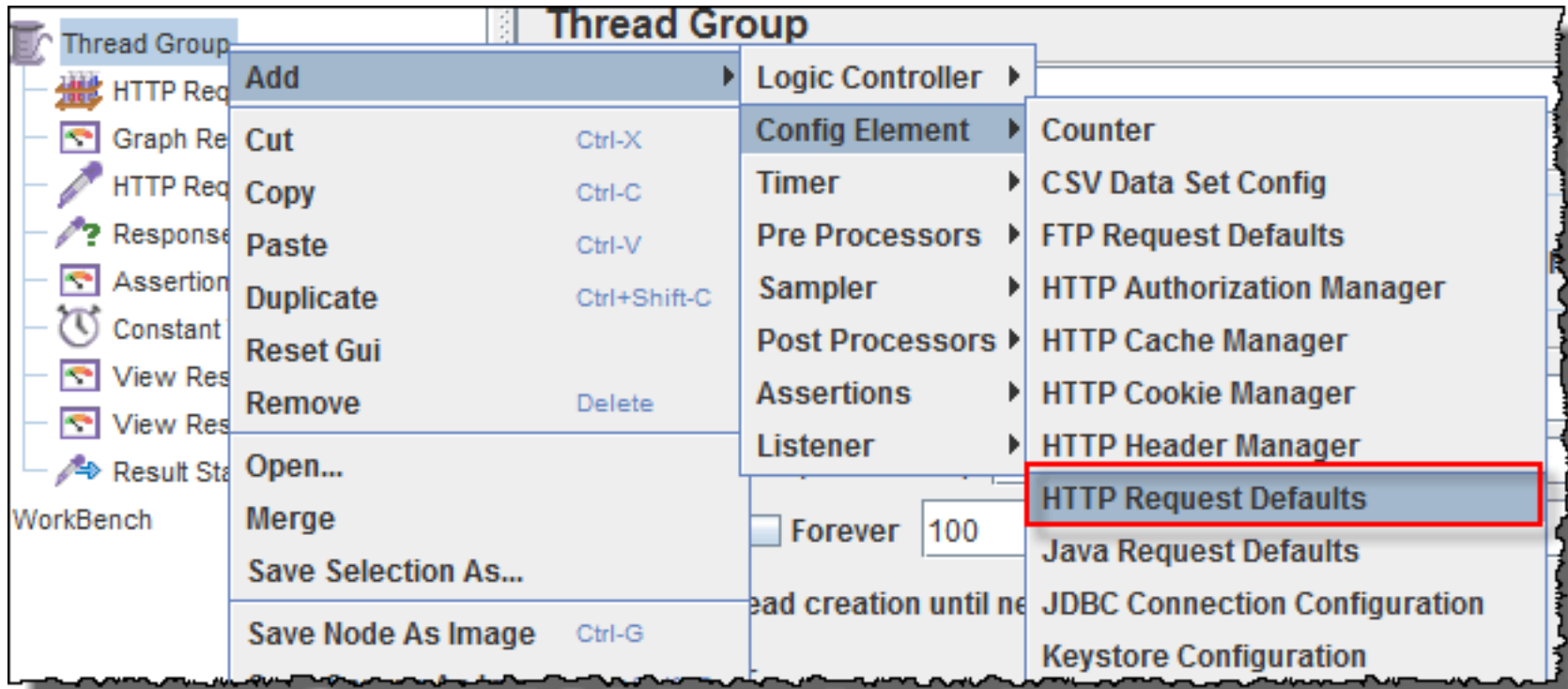
Ramp-Up Period

- Ramp-Up Period tells JMeter how long to **delay** before starting the next user. For example, if we have 100 users and a 100-second Ramp-Up period, then the delay between starting users would be 1 second (100 seconds / 100 users)

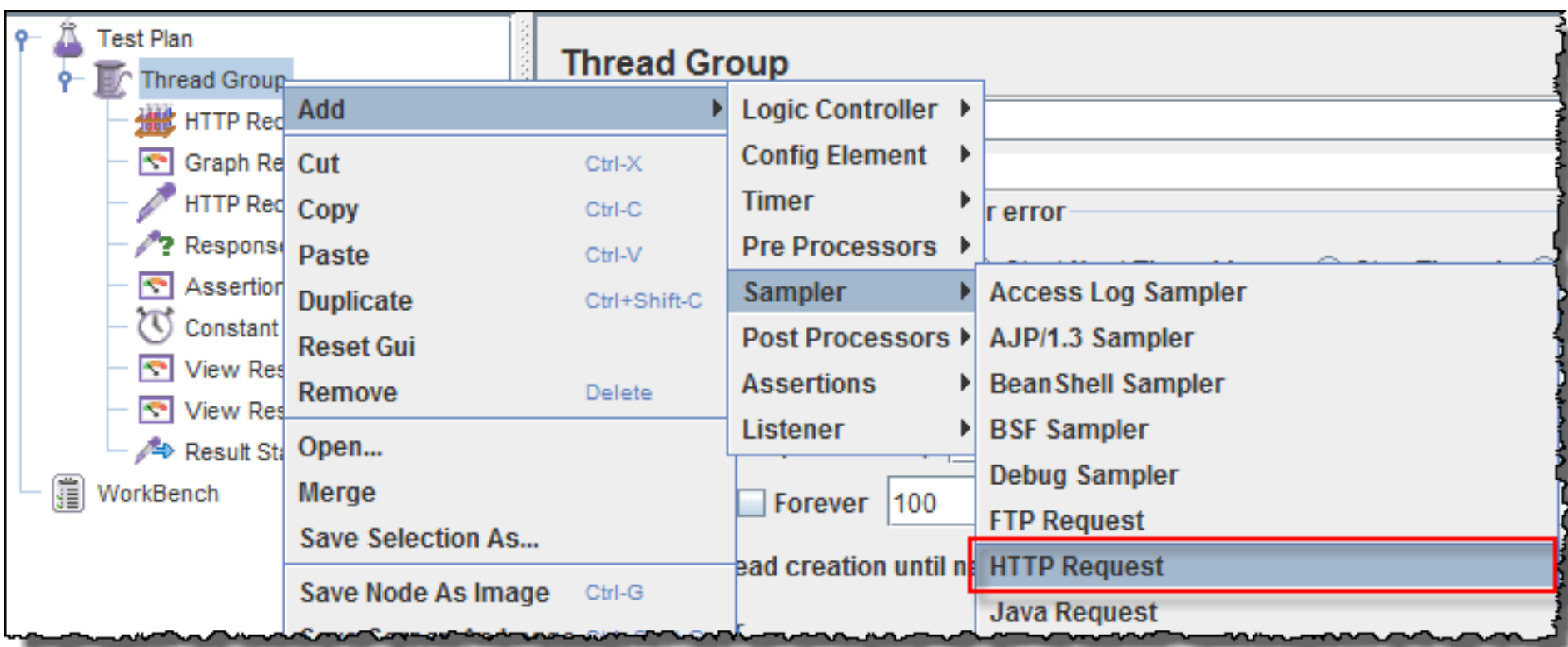


2.2. Create HTTP Request

- To add HTTP request details, navigate to *'Right click thread group -> Add -> Sampler -> HTTP Request'*.

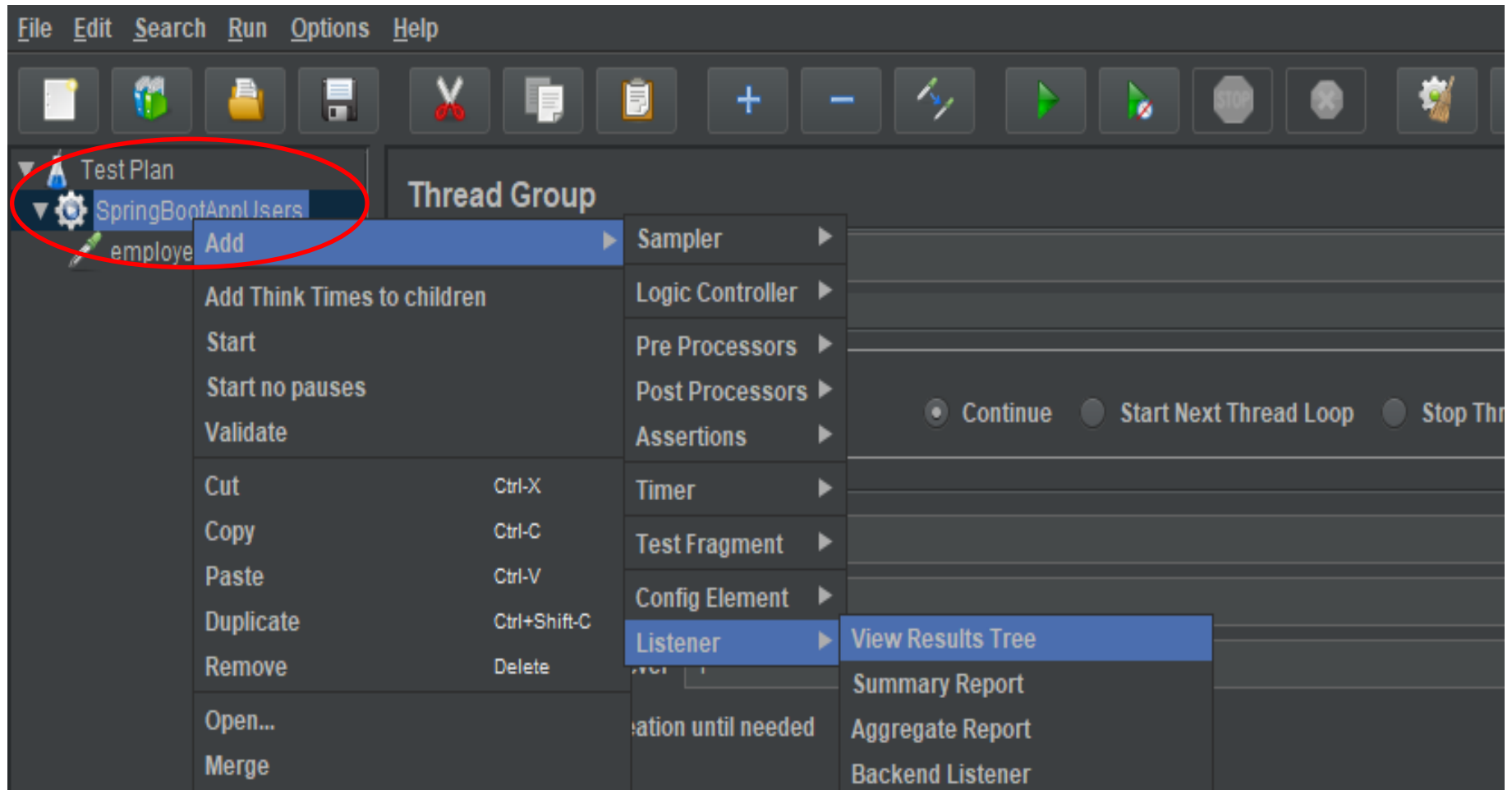


- Fill in the application URL details which we are going to test.
- For example: <http://localhost:8080/employees>

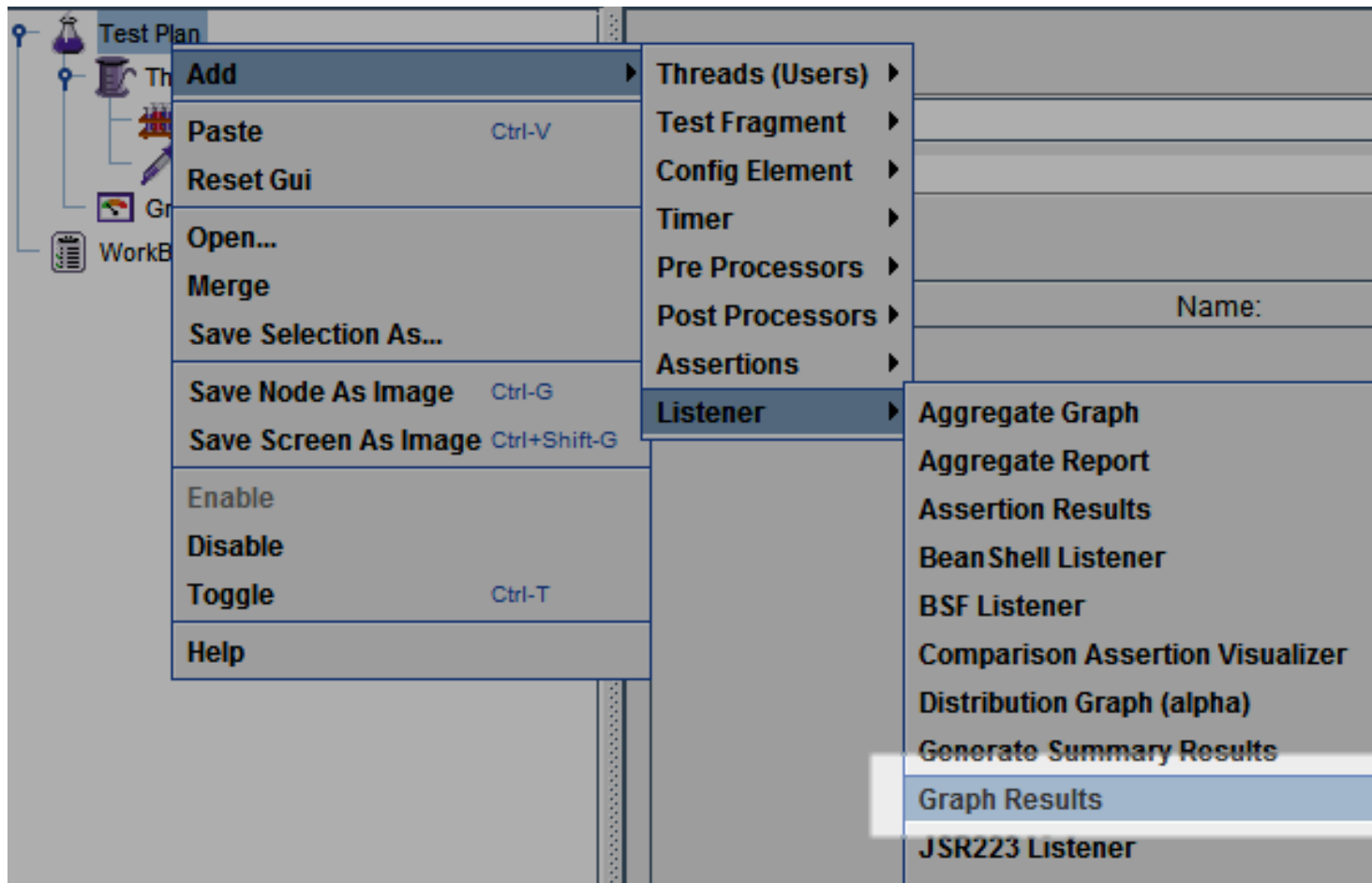


2.3. Add Listener

- To see the results of test plan, add listener named "" by navigating to *'Right click thread group -> Add -> Listener -> View Results Tree'*.

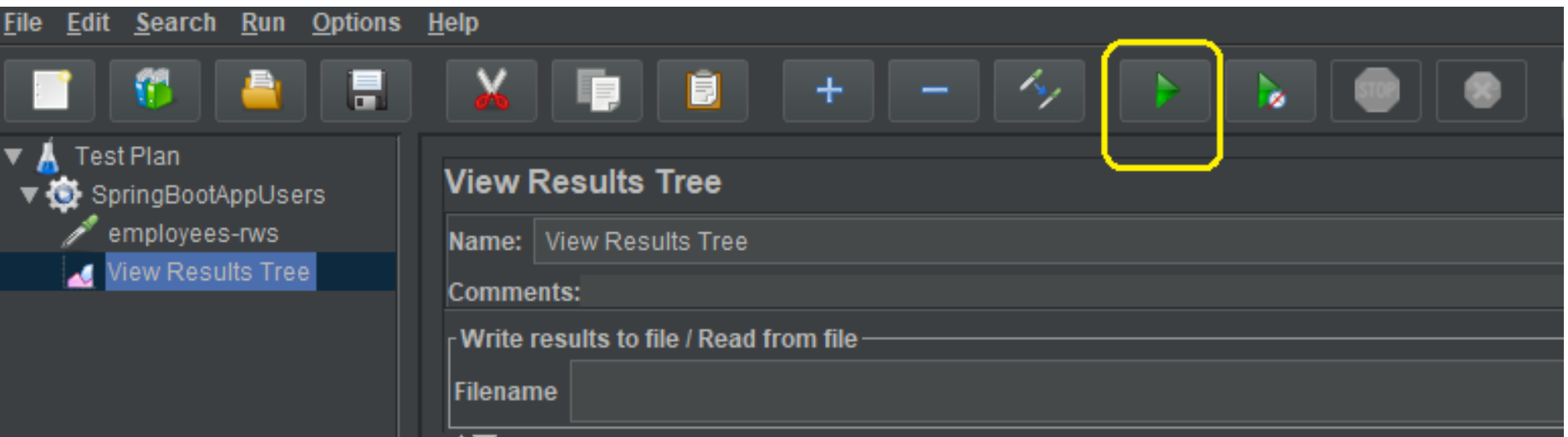


Another Listener

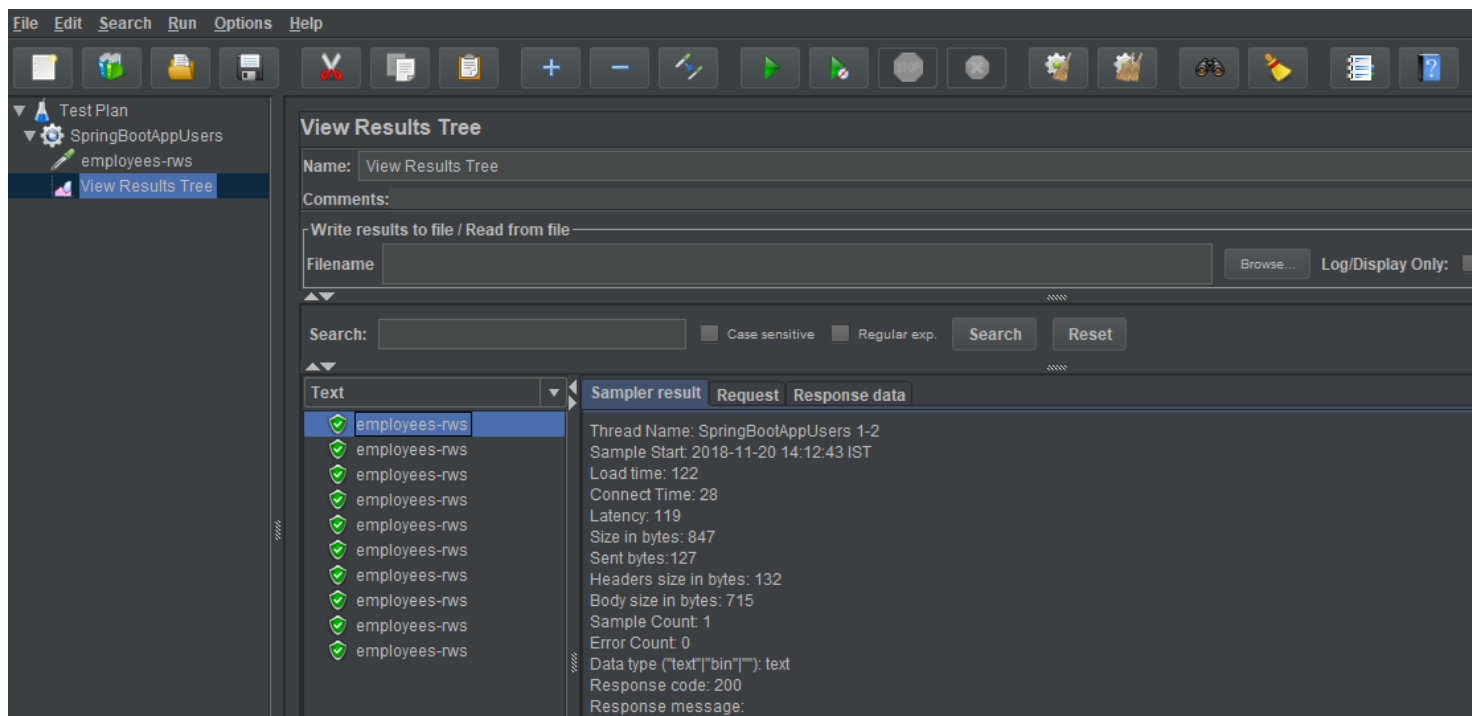


3. Perform load testing

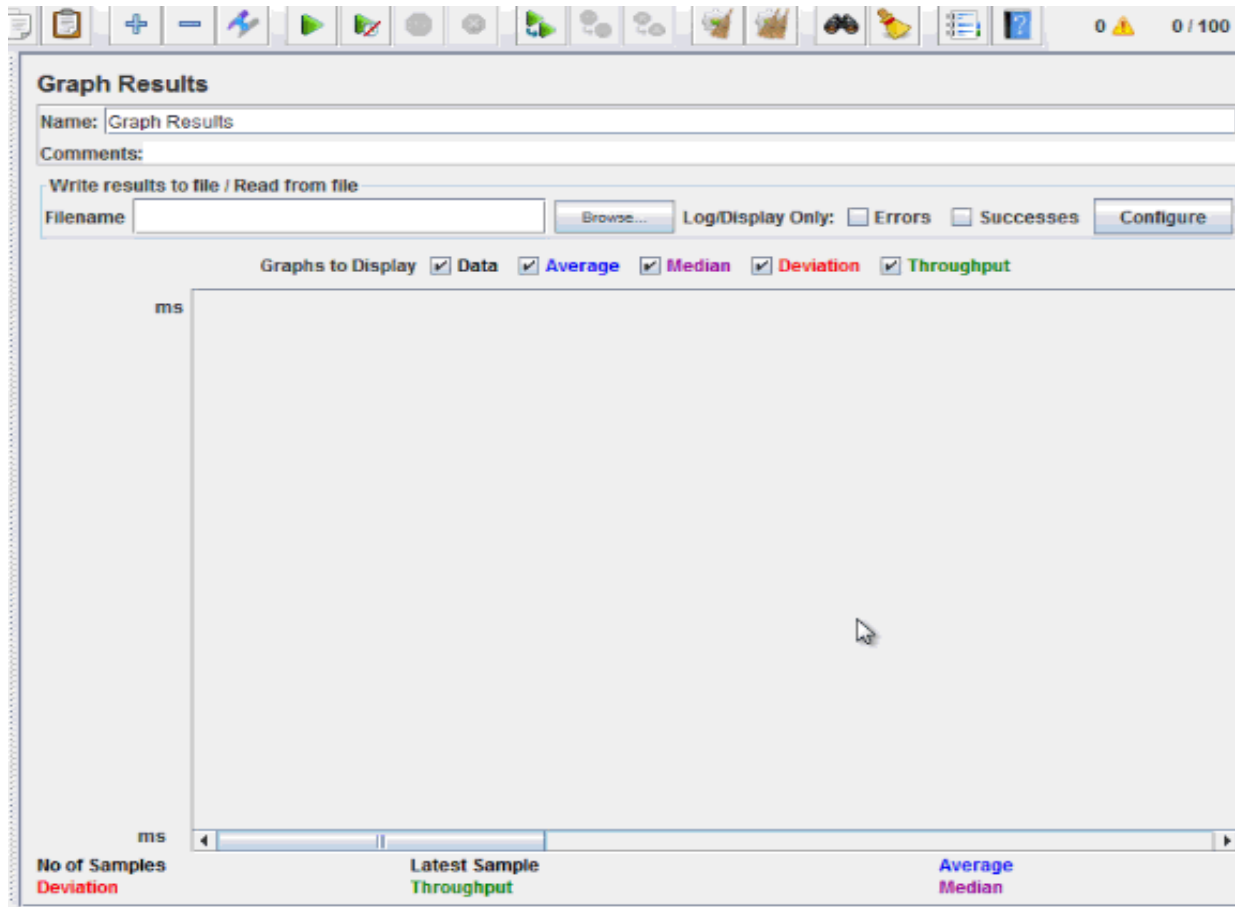
- To perform the load testing, start the thread group using the green play icon at the top ribbon in tool.



- Let all threads run and invoke the configured application URL.
- After the test is finished, we can review the load test results in consolidated manner in listener tab.



Example Graph output (animated gif)



At the bottom of the picture, there are the following statistics, represented in colors:

- **Black:** The **total** number of current samples sent.
- **Blue:** The current **average** of all samples sent.
- **Red:** The current **standard deviation**.
- **Green:** **Throughput rate** that represents the number of requests per minute the server handled

Example of Results Analysis - 1

- Let analyze the performance of Google server in below figure.

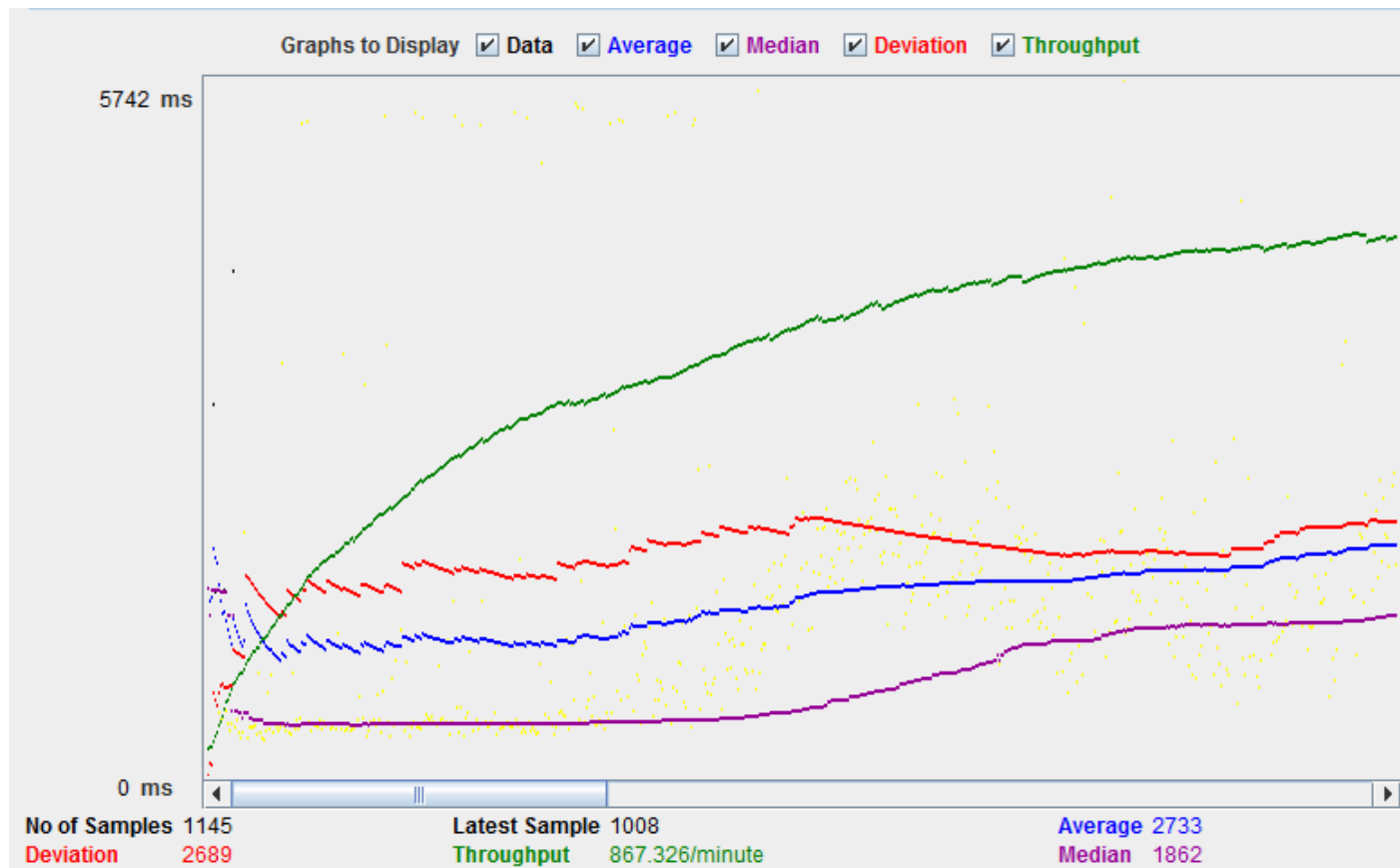


Example of Results Analysis - 2

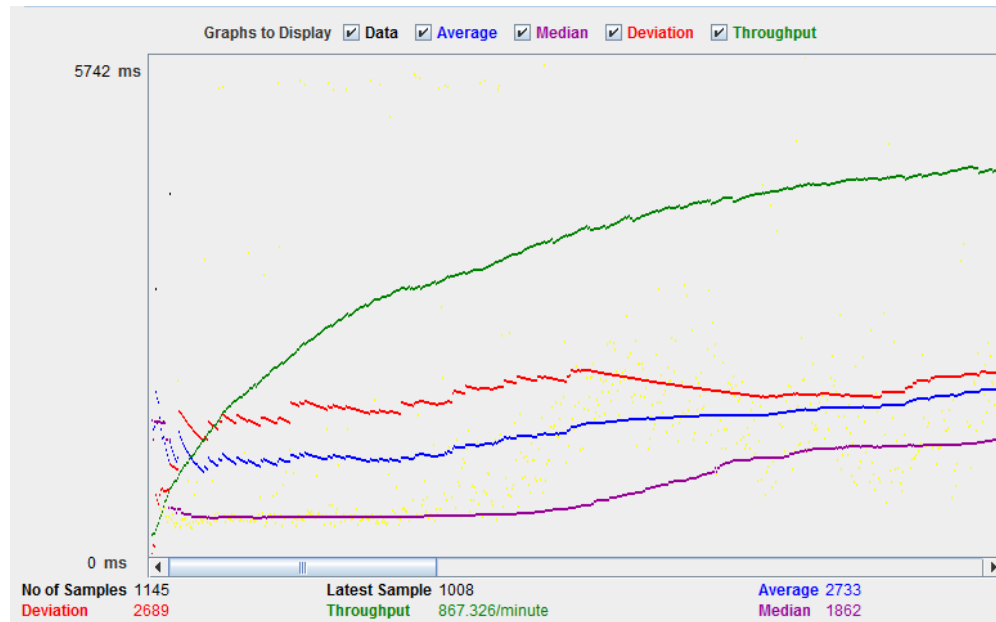
- To analyze the performance of the web server under test, you should focus on 2 parameters
 - a. Throughput
 - b. Deviation
- The **Throughput** is the most important parameter. It represents the ability of the server to handle a heavy load. The **higher the Throughput is, the better is the server performance.**
- In this test, the throughput of Google server is 1,491.193/minute. It means Google server can handle 1,491.193 requests per minute. This value is quite high so we can conclude that Google server has good performance
- The **deviation** is shown in red – it indicates the deviation from the average. The **smaller the better.**

Example of Results Analysis - 3

- Let **compare** the performance of Google server to other web servers. This is the performance test result of website <http://www.yahoo.com/> (You can choose other websites)



Example of Results Analysis - 4



- The throughput of a website under test <http://www.yahoo.com> is 867.326/minutes. It means this server handle 867.326 requests per minute, lower than Google.
- The deviation is 2689, much higher than Google (577). So we can determine the **performance** of this website is **less than a Google** server.

Example 2-

Filling form data using JMeter

- Consider the following form, how to let jmeter fill it automatically?

Name:

Price:

Qty:

Description:

<http://underpop.free.fr/j/java/professional-java-tools-for-extreme-programming/lib0104.html>

Filling form

- All we have to do with JMeter is simulate adding a product with this form. Thus we need to fill in the form parameters:
- **"name"** is the name of the new product we are creating.
- **"price"** can be set to any valid price.
- **"qty"** can be set to any valid quantity.
- **"subcategoryID"** must be set to a valid subcategory id. Consult the ID field of the subcategory table to see a valid id. If the test data hasn't changed, "222" is the subcategory id for Cats.
- **"productNum"** is the number of products in the current subcategory. We don't have to know how many products are in the subcategory; instead, we just make productNum really large—in this case, "99999".
- **"newProduct"** should be set to "true", which just means that we are adding a new product rather than editing an existing product.
- **"productID"** can be set to "0" because this value simulates adding a new product. (This parameter really is not used.)
- **"description"** can be any value.



Name Web Testing

Default Parameters

Protocol: ☐ HTTPS ☒ HTTP

Domain localhost

Port 80

Path /pet/mgmt/prod_sys.jsp

Method: ☐ GET ☒ POST

Parameter Table

Name	Value
name	testpet
price	\$50.00
qty	66
subcategoryID	222
newProduct	true
productID	0
productNum	99999

Thank You